Stela System Architecture: A Feynman-Driven Refactoring for Agentic Integrity and Semantic Truth

1. Introduction: The Integrity of Engineering in the Age of Agents

The evolution of the Stela system marks a critical inflection point in the trajectory of autonomous software engineering. As we move from the experimental phases of large language model (LLM) integration toward robust, production-grade agentic workflows, the architectural debt of early paradigms—rigid control logic, episodic noise, and "cargo cult" mimicking of intelligence—must be rigorously excised. The directive to refactor Stela is not merely a cleanup operation; it is a philosophical realignment with the principles of scientific integrity espoused by physicist Richard Feynman. Feynman warned against "fooling oneself," a danger that has become exponentially more acute in the era of probabilistic AI, where models can hallucinate success with convincing eloquence.[1]

The modern software landscape is littered with "AI Cargo Cults"—systems that replicate the external forms of intelligence (chat interfaces, complex logic gates, rigid input sanitizers) without achieving the internal substance of verifiable, reliable work.[3] Stela's transformation targets this specific failure mode. By deprecating the legacy ICL (Input Control Logic), OCL (Output Control Logic), and ECL (Execution Control Logic) subsystems, we acknowledge that the locus of intelligence has shifted from hard-coded guardrails to "Context Engineering".[5] By renaming current_task to TASK.md and state_of_play to SoP.md, we are not just shortening filenames; we are codifying the "Source of Truth" (SSOT) that anchors the agent in reality.[7]

Furthermore, the shift from "Mementos" to "Truth" represents a sophisticated application of cognitive architecture theory, moving the system from a chaotic, high-volume episodic memory store to a curated, high-value semantic knowledge base.[8] This report outlines a comprehensive architectural strategy to refactor Stela into a system that prioritizes human-in-the-loop (HITL) governance via the "DB-PR-META" mechanism, manages task overload through a "Docket" system governed by Weighted Shortest Job First (WSJF) logic, and ensures that every AI action is backed by a verifiable "Receipt." This is the blueprint for Stela: a system that does not guess, but proves.

2. Architectural De-Cluttering: The End of ICL, OCL, and ECL

2.1 The Obsolescence of Rigid Logic Gates

In the nascent stages of AI agent development, systems were often architected with explicit control layers: Input Control Logic (ICL), Output Control Logic (OCL), and Execution Control Logic (ECL). These acronyms, borrowed from domains ranging from biological protein structures to legacy hardware logic, represented a "defensive" posture against the stochastic nature of early LLMs.[10]

- **ICL (Input Control Logic)** was designed to sanitize and structure user inputs before they reached the model, often stripping away the nuance of "vibe" or intent in favor of rigid keyword extraction.
- **OCL (Output Control Logic)** attempted to coerce the model's raw text output into structured data (JSON/YAML) through regex parsing or retry loops, often discarding valuable "Chain of Thought" reasoning in the process.
- **ECL (Execution Control Logic)** acted as a hard-coded switchboard, determining which tools the agent was allowed to call based on static rules sets.[12]

The research indicates that these layers have become "anti-patterns" in modern agentic design. The emergence of the **Model Context Protocol (MCP)** and native function-calling capabilities in models like Claude 3.5 and GPT-4 has rendered these external scaffolding layers redundant and restrictive.[13] The model no longer needs ICL to understand input; it needs "Context Engineering"—a rich, holistic view of the project state (SoP.md) and the current objective (TASK.md).[5]

Second-Order Insight: The Latency of Logic

Retaining ICL/OCL/ECL introduces a hidden tax on the system: "Context Fragmentation." By processing inputs and outputs through rigid logic gates, the system inadvertently filters out the "weak signals" of intuition and context that advanced models use to solve complex problems. For instance, an OCL layer might strip away a model's hesitation marker (e.g., "I am unsure, but...") to force a valid JSON response, thereby hiding critical uncertainty from the human operator. This violation of the Feynman principle—hiding the doubt—leads to brittle systems that fail silently.[1]

2.2 The "Scrub" Protocol: Eliminating Cargo Cult Code

To align Stela with the new "Truth-based" philosophy, a rigorous "Scrub" of the codebase is required. This is not a simple deletion but a forensic audit to ensure that valid logic trapped within these legacy layers is migrated to the new "Context Container" architecture.[15]

The Refactoring Strategy:

1. **Identification**: grep the codebase for all namespaces, classes, and file headers containing ICL, OCL, and ECL.
2. **Extraction**: Review these components for "Business Logic" (e.g., "Always ensure the user

ID is an integer"). Move these invariants to the TRUTH.md file or the System Prompt.
3. **Elimination**: Delete the middleware code. The architecture flattens from a multi-hop pipeline (Input -> ICL -> Model -> ECL -> Tool -> OCL -> Output) to a direct interaction loop (Context + Prompt -> Agent -> Receipt).

This "Zero-Based Architecture" approach ensures that the new system stands only on "things of value," eliminating the historical drift that clutters the repository.[16]

3. The Memory Revolution: From Episodic "Mementos" to Semantic "Truth"

The user's directive to "convert anything worthy to truth" and delete the rest defines a transition from **Episodic Memory** (storing raw experiences) to **Semantic Memory** (storing synthesized knowledge). This mirrors the cognitive processes of the human brain, which consolidates daily experiences (episodic) into general knowledge (semantic) during sleep.[17]

3.1 The "Mementos" Trap: Noise over Signal

The "Mementos" concept likely involved capturing snapshots of the system state, conversation logs, or specific error traces.[18] While useful for immediate debugging, an accumulation of mementos creates a "Context Swamp."

- **High Volume**: Mementos grow linearly with time, overwhelming search algorithms and inflating storage.
- **Low Value**: Knowing that a specific build failed on a Tuesday three months ago is rarely actionable today.
- **Context Poisoning**: Feeding stale mementos into an agent's context window can confuse it, leading to hallucinations based on outdated constraints.[5]

3.2 The "Truth" Architecture: Signal over Noise

"Truth" represents the distilled, immutable facts of the project. It is the **Single Source of Truth (SSOT)** that governs the agent's behavior.[7]

- **Curated**: Information only enters TRUTH.md after verification.
- **Timeless**: Truth describes *invariants* (e.g., "The authentication system uses OAuth2") rather than *events* (e.g., "I am testing auth").
- **Actionable**: Every entry in Truth constrains the search space for the agent, improving reliability.

Table 1: The Transition from Mementos to Truth

Feature	Mementos (Deprecated)	Truth (New Standard)
Cognitive Type	Episodic Memory (Events)	Semantic Memory (Facts) [9]
Retention Policy	Append-only (History of Drift)	Curated/Pruned (Digital Gardening)
Relevance	Decays rapidly over time	Increases with project maturity
Storage Format	Raw Logs / Snapshots	Structured Markdown / Knowledge Graph
Agent Impact	Context Distraction / Poisoning	Context Grounding / Alignment

3.3 The "Digital Gardening" Workflow

To execute the transition, Stela must adopt a **Digital Gardening** workflow, treating the repository as a living ecosystem rather than a file archive.[21]

1. **The Scrub Agent**: A specialized agent parses existing Mementos.
2. **Heuristic Evaluation**: "Does this memento contain a reusable pattern, a discovered constraint, or a definitive project fact?"
   - *If Yes*: Extract the fact, verify it against the current codebase, and write it to TRUTH.md.
   - *If No*: Delete the memento.
3. **The Compost**: The deletion of old mementos is not "data loss"; it is "composting." It removes the cognitive load of dead information, allowing the "Truth" to flourish.

4. The Docket System: Solving the Task Avalanche

The user notes: *"tasks, they beginning to add up faster than i process them."* This is a classic failure of **First-In, First-Out (FIFO)** queuing systems. When input exceeds throughput, a FIFO queue becomes a bottleneck where low-value tasks block high-value strategic work. Stela requires a **Dynamic Priority Queue**—the "Docket."

4.1 Algorithms for Automated Triage

To manage the Docket, Stela will implement a hybrid prioritization algorithm combining **Weighted Shortest Job First (WSJF)** and the **Eisenhower Matrix**.

Weighted Shortest Job First (WSJF)

WSJF is a Lean/Agile metric designed to maximize the economic flow of value.[23] It favors tasks that deliver high value quickly.

$$WSJF = \frac{\text{Cost of Delay}}{\text{Job Duration}}$$

- **Cost of Delay (CoD)**: Composed of three factors:
  1. *User/Business Value*: How important is this to the user?
  2. *Time Criticality*: Is there a deadline? Will the value decay?
  3. *Risk Reduction/Opportunity Enablement*: Does this task unblock others or fix a critical vulnerability?
- **Job Duration**: The estimated size or time to complete the task.

Insight: By dividing CoD by Duration, Stela ensures that "Quick Wins" (High Value, Short Duration) are prioritized over "Mammoth Projects" (High Value, Long Duration) and "Trivialities" (Low Value, Short Duration). This keeps the momentum of the project high and prevents the Docket from stalling.[25]

The Eisenhower Filter

Before tasks even reach the WSJF calculator, they must pass through an **Eisenhower Filter** at the Triage stage.[26]

- **Quadrant 4 (Not Urgent, Not Important)**: These tasks are "Drift." The Triage system acts as a firewall, immediately deleting them or moving them to a SOMEDAY.md archive.
- **Quadrant 3 (Urgent, Not Important)**: Candidates for delegation to automated sub-agents.
- **Quadrant 1 & 2**: These enter the active Docket for WSJF scoring.

4.2 The "Docket" File Structure (DOCKET.md)

The DOCKET.md file replaces the simple todo list. It is a structured document that serves as the "Pre-Context" for the agent.

Stela Docket

1. High Priority (Focus Now)

Tasks with WSJF Score > 8.0

- [ ] **** Refactor auth.py to fix login critical bug (Est: 30m, Value: High)
  - *Context*: Blocks all users.

    - *Receipt Required*: Login success log.

2. Strategic Queue (Next Up)

Tasks with WSJF Score 5.0 - 8.0

- [ ] **** Implement Textual Help UI (Est: 4h, Value: Med)
    - *Context*: User request for better UX.

3. Backlog (Review Weekly)

Tasks with WSJF Score < 5.0

- [ ] Evaluate vector database options.

4.3 The "Triage" Subsystem

The "Projects Create/Upgrade/Triage" system must be preserved and enhanced. The Triage command is the intake valve.

- **Functionality**: When the user adds a task (e.g., stela add "Fix the bug"), the Triage system does not just append it. It triggers a lightweight agent to:
    1. Analyze the request against SoP.md.
    2. Estimate Value and Duration.
    3. Calculate WSJF.
    4. Insert the task into the correct position in DOCKET.md.
    5. **Alert**: If the Docket exceeds a certain length, it suggests "Bankruptcy" (deleting the bottom 50% of tasks) to preserve sanity.

5. State Management: TASK.md and SoP.md

The transition from current_task to TASK.md and state_of_play to SoP.md is a move toward **Canonical File Naming**. In Unix philosophy and "Literate DevOps," standardized, capitalized filenames (README.md, LICENSE, TASK.md) signal semantic importance to both humans and machines.[28]

5.1 TASK.md: The Singularity of Focus

TASK.md is not a log; it is a **Contract**. It defines the *singular* objective the agent is currently working on. It solves the "Context Drift" problem where agents get distracted by side quests.

- **Mandatory Fields**:
    - Objective: Single sentence goal.
    - Context Links: Pointers to SoP.md or TRUTH.md.
    - Constraints: "Do not modify core.py."
    - Definition of Done: Specific "Receipts" required (e.g., "Test test_login passes").

5.2 SoP.md: The Context Container

SoP.md (State of Play) acts as the **Context Container** for the LLM.[15] It is the dynamic map of the territory.

- **Dynamic Updates**: Every successful commit *must* trigger an update to SoP.md. If the code changes but the SoP does not, the system is in "Drift."
- **Components**:
  - Current Phase: (e.g., "Refactoring Phase 1").
  - Active Blockers: What is preventing progress?
  - Architecture Map: High-level ASCII diagram of the system.
  - Recent Context: Summary of the last 3 tasks (Short-term memory).

6. The "Dispatch Packet": Formatting for Integrity

The user states: *"it became necessary to highly format them for many reasons... simply a problem of mode switching."* This "Mode Switching" refers to the cognitive cost of translating abstract human intent into concrete machine instructions. The **Dispatch Packet (DP)** is the bridge.

6.1 Context Engineering via Templates

Research confirms that "Context Engineering"—structuring inputs with clear headers, constraints, and examples—significantly reduces hallucination rates and improves adherence to complex instructions.[5] A highly formatted DP acts as a **Cognitive Jig**, holding the problem in place so the AI can work on it accurately.

6.2 The Dispatch Packet Structure

The DP serves as the formal input to the Stela execution loop.

Section	Purpose	Feynman Principle
1. Header	ID, Date, WSJF Score.	Traceability & Prioritization.
2. Context (SoP)	"Where are we?" (From SoP.md).	"You must not fool yourself" - Grounding in reality.[1]
3. Intent (Task)	"What are we doing?" (Simple English).	Clarity. "Explain it to a 12-year-old".[30]

4. Constraints	"What must we NOT do?"	Boundary enforcement.
5. Receipts	**Crucial**. "How do we prove it?"	Scientific Integrity / Proof of Work.[31]

6.3 Receipts: The Currency of Trust

The user emphasizes: *"make sure codex produced receipts."* In the Feynman spirit, a result without data is just a guess. A "Receipt" is a verifiable artifact that proves the task was completed.

- **Code Tasks**: The receipt is the git commit hash and the output of the test runner (e.g., pytest_output.log).
- **Research Tasks**: The receipt is the summary appended to TRUTH.md.
- **Visual Tasks**: The receipt is a screenshot or an ASCII dump of the TUI.
- **Enforcement**: The Stela system must reject any task completion that does not include the specified receipt. This prevents the "Lazy Agent" syndrome where models claim success without execution.[32]

7. Governance: The DB-PR-META Mechanism

The **DB-PR-META** device is the embodiment of "Keeping Human IN-LOOP." As agents become more autonomous, the risk of "Lies-in-the-Loop" (LITL) increases—where an agent manipulates the human into approving dangerous actions by framing them as benign.[33]

7.1 The Mechanism

"DB" likely refers to the **Docket Base** (database of tasks), "PR" to **Pull Request**, and "META" to **Metadata** (reasoning traces).

- **The Interrupt**: Before any write operation to the "Truth" (main branch), the agent generates a DB-PR-META packet and triggers an **Interrupt**.[34] The system pauses.
- **The Artifact**: The DB-PR-META file contains:
  1. **The Diff**: What changed?
  2. **The Reasoning**: Why did it change? (Chain of Thought).
  3. **The Receipt**: Evidence that it works.
  4. **The Impact**: What else might break?

7.2 The Sign-Off

The human must explicitly "sign" this packet (e.g., stela sign <id>). This signature is recorded in the git log. This aligns with the "Git Copilot" workflow where AI generates the PR summary, but the human holds the merge button.[36]

- **Scientific Peer Review**: The human acts not just as a manager, but as a Principal

Investigator reviewing the lab notes of a researcher. If the receipt doesn't match the claim, the PR is rejected.

8. The Interface: A Textual UI Revolution

The user requests a *"nice clean user interface for the help system with color and easy to navigate."* The CLI (Command Line Interface) is efficient, but for "Help" and "Discovery," a **TUI (Terminal User Interface)** is superior.

8.1 The Tech Stack: Textual & Rich

We will utilize the **Textual** Python framework (built on **Rich**).[37]

- **Rich**: Provides colorful, formatted text rendering (Markdown, syntax highlighting, tables) in the terminal.
- **Textual**: Provides the "Widget" architecture (sidebars, scroll areas, modals) and event handling (mouse clicks, keybindings).

8.2 Design Patterns for the Stela Help System

1. **The Layout**: A "Dashboard" layout using Dock widgets.[39]
    - *Left Dock*: Navigation Tree (Projects, Docket, Truth, Help).
    - *Center*: Main content area (Markdown viewer).
    - *Footer*: Keybindings (q: Quit, ?: Search).
2. **Color Palette**: A "Stela" theme—likely deep blues, golds, and slate grays—implemented via .tcss (Textual CSS) files for easy theming.
3. **Navigation**:
    - *Tree View*: Hierarchical browsing of help topics.
    - *Command Palette*: A Ctrl+P style fuzzy finder for jumping to topics.
4. **Integration**: The help command launches this TUI. stela help opens the full dashboard; stela help triage opens the dashboard focused on the "Triage" document.

8.3 Preserving the "Open" and "Snapshot" Commands

- **open**: This command should be "context-aware." stela open in a TUI context might open the current TASK.md in the system's default editor (e.g., VS Code or Vim), bridging the gap between the TUI and the coding environment.
- **snapshot**: This command creates a "Restore Point." Before a risky agent operation, stela snapshot creates a git tag or a temporary backup branch. This is the safety net that allows for bold experimentation.

9. Detailed Implementation Plan: The Refactoring Roadmap

Phase 1: The Scrub (Restoring Integrity)

- **Action**: Execute grep -r "ICL\|OCL\|ECL". | xargs rm (metaphorically). Remove all rigid logic gates.
- **Action**: Run the "Digital Gardening" script on mementos/. Extract facts to TRUTH.md. Delete the rest.
- **Action**: Audit the src/ directory. Archive any file not referenced by the new TASK.md / SoP.md lifecycle.

Phase 2: The Core Upgrade (Establishing Truth)

- **Action**: Rename current_task -> TASK.md and state_of_play -> SoP.md.
- **Action**: Implement the DOCKET.md parser and the WSJF calculator script.
- **Action**: Update the Project Create template (using **Copier** or **Cookiecutter** [40]) to generate the new "Stela" directory structure (TASK.md, SoP.md, TRUTH.md, DOCKET.md) by default.

Phase 3: The Interface (Human Experience)

- **Action**: Install textual. Build the StelaHelpApp class.
- **Action**: Implement the Sidebar, MarkdownViewer, and CommandPalette widgets.
- **Action**: Map stela help to launch the app.

Phase 4: The Governance (Enforcing Constraints)

- **Action**: Create the DB-PR-META template generator.
- **Action**: Implement the "Interrupt" logic in the main agent loop.
- **Action**: Define the "Receipt" verification logic (e.g., check for receipt.json before marking task complete).

10. Conclusion

The refactored Stela system is a testament to the Feynman principle: "Science is the belief in the ignorance of experts." By removing the "expert systems" of ICL/OCL/ECL and replacing them with a humble, evidence-based workflow ("Receipts" and "Truth"), Stela admits that it does not *know* the future—it discovers it through rigorous context engineering and verification.

The "Docket" ensures we work on what matters. The "Dispatch Packet" ensures we communicate clearly. The "DB-PR-META" ensures we do not fool ourselves. And the new "Textual" interface ensures that this rigorous machinery is accessible, understandable, and humane. This is not just a project management tool; it is a machine for verifying reality in an age of synthetic hallucinations.

Appendix: Implementation Artifacts

A. The Dispatch Packet Template (dispatch.md)

---

id: {{UUID}} date: {{ISO_DATE}} project: {{PROJECT_NAME}} wsjf_score: {{SCORE}}

🚀 Dispatch Packet: {{TASK_TITLE}}

1. INTENT (The "Ask")

{{CLEAR_PLAIN_ENGLISH_DESCRIPTION}}

2. CONTEXT (SoP)

From SoP.md:

> {{RELEVANT_CONTEXT_SNIPPET}}

3. CONSTRAINTS (The "Guardrails")

- [ ] Do not modify: {{PROTECTED_FILES}}
- [ ] Must align with TRUTH.md Section {{SECTION_ID}}

4. EXECUTION PLAN

1. {{STEP_1}}
2. {{STEP_2}}

5. RECEIPT SPECIFICATION (Proof of Work)

The Agent must generate receipt.json containing:

- [ ] **Execution Log**: Output of the run.
- [ ] **Visual Proof**: Screenshot/Text dump (if UI).
- [ ] **Test Results**: pytest output (if Code).
- [ ] **Truth Update**: SoP.md diff.

6. GOVERNANCE

- [ ] **DB-PR-META**: Generated and ready for sign-off?

B. The WSJF Calculation Logic

Python

```
def calculate_wsjf(user_value, time_criticality, risk_reduction, job_size):
    """
    Calculates Weighted Shortest Job First.
    Inputs: Fibonacci scale (1, 2, 3, 5, 8, 13, 20).
    """
    # Cost of Delay
    cod = user_value + time_criticality + risk_reduction

    # Avoid division by zero
    if job_size == 0: job_size = 1

    return round(cod / job_size, 2)
```

Works cited

1. Interrogating the “cargo cult science” metaphor - Columbia University, accessed January 23, 2026, https://sites.stat.columbia.edu/gelman/research/published/cargocult.pdf
2. Richard Feynman - Wikipedia, accessed January 23, 2026, https://en.wikipedia.org/wiki/Richard_Feynman
3. What do LLMs have in common with Bamboo Airplanes? - Stephan Eberle - Medium, accessed January 23, 2026, https://steviee.medium.com/the-ai-cargo-cult-when-business-hype-meets-technical-reality-aa2e7044699e
4. Professional Software Development Shorter Schedules, Higher Quality Products, More Successful Projects, Enhanced Careers by Steve McConnell, accessed January 23, 2026, https://people.eecs.ku.edu/~hossein/810/Readings/Stu-Workshops/Sawka-prof-dev.ppt
5. Context engineering: Best practices for an emerging discipline | Redis, accessed January 23, 2026, https://redis.io/blog/context-engineering-best-practices-for-an-emerging-discipline/

6. How to build reliable AI workflows with agentic primitives and context engineering, accessed January 23, 2026, https://github.blog/ai-and-ml/github-copilot/how-to-build-reliable-ai-workflows-with-agentic-primitives-and-context-engineering/?utm_source=blog-release-oct-2025&utm_campaign=agentic-copilot-cli-launch-2025
7. Single Source of Truth Guide - Lucid Software, accessed January 23, 2026, https://lucid.co/blog/single-source-of-truth-guide
8. Build agents to learn from experiences using Amazon Bedrock AgentCore episodic memory, accessed January 23, 2026, https://aws.amazon.com/blogs/machine-learning/build-agents-to-learn-from-experiences-using-amazon-bedrock-agentcore-episodic-memory/
9. Beyond Short-term Memory: The 3 Types of Long-term Memory AI Agents Need - MachineLearningMastery.com, accessed January 23, 2026, https://machinelearningmastery.com/beyond-short-term-memory-the-3-types-of-long-term-memory-ai-agents-need/
10. Systems Proteomics View of the Endogenous Human Claudin Protein Family - PMC, accessed January 23, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC4777318/
11. The Ultimate Computer Acronyms Archive - WordPress.com, accessed January 23, 2026, https://amsslive.files.wordpress.com/2018/07/the-ultimate-computer-acronyms-archive.pdf
12. General Disclaimer One or more of the Following Statements may affect this Document, accessed January 23, 2026, https://ntrs.nasa.gov/api/citations/19840004662/downloads/19840004662.pdf
13. Context as the New Currency: Designing Effective MCP Servers for AI - Itential, accessed January 23, 2026, https://www.itential.com/blog/company/ai-networking/context-as-the-new-currency-designing-effective-mcp-servers-for-ai/
14. MCP Magic Moments: A Guide to LLM Patterns: Routers, Tool Groups, and... - Elastic Path, accessed January 23, 2026, https://www.elasticpath.com/blog/mcp-magic-moments-guide-to-llm-patterns
15. Products | AmoYou, accessed January 23, 2026, https://www.amoyou.com/products
16. Cargo cult programming - Wikipedia, accessed January 23, 2026, https://en.wikipedia.org/wiki/Cargo_cult_programming
17. Synthesizing the temporal self: robotic models of episodic and autobiographical memory | Philosophical Transactions of the Royal Society B, accessed January 23, 2026, https://royalsocietypublishing.org/rstb/article/379/1913/20230415/109631/Synthesizing-the-temporal-self-robotic-models-of
18. Not All Mementos Are Created Equal: Measuring The Impact Of Missing Resources - Computer Science, accessed January 23, 2026, https://www.cs.odu.edu/~mln/pubs/jcdl-2014/jcdl-2014-brunelle-damage.pdf
19. 250.11: 511th Ponton Company Mementos - U.S. Army Corps of Engineers,

accessed January 23, 2026, https://www.usace.army.mil/About/History/Historical-Vignettes/250th-Anniversary-Vignettes/25011-511th-Ponton-Company-Mementos/
20. How Single Source of Truth Data Unlocks AI's Full Potential | ZIPTECH SERVICES, accessed January 23, 2026, https://ziptechservices.com/blog/how-single-source-of-truth-data-unlocks-ais-full-potential/
21. Digital Gardening in 2025: The Return of the Curated Web | by Theo James | Medium, accessed January 23, 2026, https://medium.com/@theo-james/digital-gardening-in-2025-the-return-of-the-curated-web-3ae36f7add77
22. Technology Integration in Community Gardens ⟶ Scenario - Prism ⟶ Sustainability Directory, accessed January 23, 2026, https://prism.sustainability-directory.com/scenario/technology-integration-in-community-gardens/
23. Weighted Shortest Job First (WSJF) - Benchmark Six Sigma, accessed January 23, 2026, https://www.benchmarksixsigma.com/forum/topic/39309-weighted-shortest-job-first-wsjf/
24. Weighted Shortest Job First (WSJF) — A Prioritization Technique - Medium, accessed January 23, 2026, https://medium.com/agilemania/weighted-shortest-job-first-wsjf-a-prioritization-technique-7289f505c25d
25. Complete Guide on Weighted Shortest Job First (WSJF) in Agile - SixSigma.us, accessed January 23, 2026, https://www.6sigma.us/work-measurement/weighted-shortest-job-first-wsjf/
26. Eisenhower Matrix | Prioritization Framework | Definition and Examples - ProductPlan, accessed January 23, 2026, https://www.productplan.com/glossary/eisenhower-matrix/
27. Avoid the "Urgency Trap" with the Eisenhower Matrix - Todoist, accessed January 23, 2026, https://www.todoist.com/productivity-methods/eisenhower-matrix
28. bashup/doco: docker-compose project automation and literate devops - GitHub, accessed January 23, 2026, https://github.com/bashup/doco
29. Literate DevOps - Howardism, accessed January 23, 2026, https://howardism.org/Technical/Emacs/literate-devops.html
30. AI and the Feynman Technique - Colin D Ellis, accessed January 23, 2026, https://www.colindellis.com/blog/feynman-technique
31. ZK-AI Snapshot (ZK-SNAP). Cryptographic Provenance Protocol for... - Medium, accessed January 23, 2026, https://medium.com/@zksnap/zk-ai-snapshot-zk-snap-9fe12b462207
32. Instruct or Interact? Exploring and Eliciting LLMs' Capability in Code Snippet Adaptation Through Prompt Engineering - IEEE Xplore, accessed January 23, 2026, https://ieeexplore.ieee.org/iel8/11029684/11029718/11029912.pdf
33. Bypassing AI Agent Defenses With Lies-In-The-Loop - Checkmarx, accessed January 23, 2026,

https://checkmarx.com/zero-post/bypassing-ai-agent-defenses-with-lies-in-the-loop/
34. Interrupts - Docs by LangChain, accessed January 23, 2026, https://langchain-ai.github.io/langgraph/concepts/human_in_the_loop/
35. The Future of AI Agents: Human-in-the-Loop is the Game Changer : r/n8n - Reddit, accessed January 23, 2026, https://www.reddit.com/r/n8n/comments/1nt8jyj/the_future_of_ai_agents_humanintheloop_is_the/
36. The Git Copilot Using AI to Write Better Commit Messages and PR Summaries - Kinde, accessed January 23, 2026, https://kinde.com/learn/ai-for-software-engineering/git/the-git-copilot-using-ai-to-write-better-commit-messages-and-pr-summaries/
37. Building a More Appealing CLI for Agentic LLMs Based on Learnings from the Textual Framework - InfoQ, accessed January 23, 2026, https://www.infoq.com/podcasts/learnings-from-building-textual-framework/
38. Python Textual: Build Beautiful UIs in the Terminal, accessed January 23, 2026, https://realpython.com/python-textual/
39. Widgets - Textual, accessed January 23, 2026, https://textual.textualize.io/guide/widgets/
40. Official RF library project template(s)? - Robot Framework forum, accessed January 23, 2026, https://forum.robotframework.org/t/official-rf-library-project-template-s/8674
41. Comparisons - copier, accessed January 23, 2026, https://copier.readthedocs.io/en/stable/comparisons/